

Structured Error Responses

การออกแบบ error ให้ agent คุ้นได้อย่างชาญฉลาด

Domain 2 · Task 2.2 — Claude Certified Architect

Structured Error Responses คืออะไร?

- คือหลักการออกแบบให้ **MCP tool** สื่อสาร "ความล้มเหลว" กลับไปหา agent ด้วย **โครงสร้างที่มี metadata ชัดเจน** ไม่ใช่แค่ข้อความ error แบบ generic
- ประเด็นสำคัญ: error แต่ละแบบต้องการ **วิธีกู้คืน (recovery) คนละอย่าง**

⚠ ถ้า tool บอกไม่ชัดว่าเป็น error ประเภทไหน

agent จะเลือกกลยุทธ์กู้คืนผิด — เช่น `retry` สิ่งที่ retry ไปก็ล้มเหลวเหมือนเดิม

4 ประเภทของ Error

★ แกนหลักของ Task นี้

Transient

timeout · service unavailable · rate limit — ปลายทางเข้าไม่ถึงชั่วคราวแต่ request ถูกต้อง → `isRetryable: true` ลองใหม่หลังหน่วงเวลาสั้น ๆ

Validation

request ผิดรูปแบบ — format ผิด · field ขาด · ค่าเกินช่วง → แก้ input ให้ถูกต้องแล้วลองใหม่ (`isRetryable: true`)

Business

ละเมิด policy · เกิน limit · ขัด business rule → `isRetryable: false` เพราะ "request เติม ยังไงก็ fail" agent ต้องเปลี่ยนไปทำ workflow ทางเลือก

Permission

ถูกจำกัดสิทธิ์การเข้าถึง ต้อง escalate หรือใช้ credential อื่น → `isRetryable: false`

โครงสร้าง JSON ของ Error Response

`isError` — true / false

`content` — ข้อความอ่านได้

`errorCategory` — transient / validation /
business / permission

`isRetryable` — true / false

`description` — บรีฟให้ agent ตัดสินใจต่อ

Key Concept: isRetryable

ต้องผูกกับประเภทของ error อย่าง**สม่ำเสมอ**

transient → true

validation → true

business → false

permission → false

metadata นี้ต่างหากที่ทำให้ agent คุ้นได้ถูกทาง — ไม่ใช่การเดาจากข้อความ error ล้วน ๆ

Access Failure ≠ Valid Empty Result

★ แนวคิดสำคัญที่สุดของ Domain

Access failure

tool เข้าถึงแหล่งข้อมูล**ไม่ได้เลย** (timeout · auth fail · ระบบล่ม)

→ `isError: true` , `errorCategory: transient`

Valid empty result

tool query **สำเร็จ** แต่ "ไม่พบข้อมูลที่ตรงเงื่อนไข"

→ `isError: false` , `resultCount: 0` พร้อมข้อความว่าไม่พบ match

🔑 Key Concept

"ไม่พบข้อมูล" คือคำตอบที่**สมบูรณ์และสิ้นสุด** (**conclusive**) ไม่ใช่ความล้มเหลว — การสืบสวนสองอย่างนี้ทำให้เกิด retry และ escalation ที่ไม่จำเป็น

ตัวอย่าง JSON เทียบให้เห็นภาพ

🔴 Business error — ยังซ้ำก็ผ่านไม่ได้

```
{
  "isError": true,
  "errorCategory": "business",
  "isRetryable": false,
  "description": "Refund £750 exceeds
    the £500 automatic refund limit..."
}
```

🟢 Valid empty result — จบแล้ว หยุดค้นได้

```
{
  "isError": false,
  "content": [{ "type": "text",
    "text": "No customer found
      matching email..." }],
  "resultCount": 0
}
```

สังเกตความต่าง: อันซ้าย**ไม่มี** `isError` / `errorCategory` เพราะ "ไม่ใช่ error" · อันบนบอก agent ว่า**ต้องเปลี่ยนแผน** อันล่างบอกว่า**จบแล้ว หยุดค้นได้**

Error Propagation ในระบบ Multi-Agent

★ ออกสอบบ่อย

หลักการ

subagent ต้องทำ **local recovery** สำหรับ transient failure เองก่อน (เช่น retry timeout ในระดับตัวเอง)
แล้ว "ส่งต่อ (propagate) เฉพาะ error ที่แก้ในระดับตัวเองไม่ได้" ขึ้นไปหา coordinator

⚡ เวลาส่งต่อ error

ต้องแนบ **partial results** และบอกด้วยว่า **ลองทำอะไรไปแล้วบ้าง** เพื่อให้ coordinator ตัดสินใจต่อได้

🚫 ไม่ใช่กลืน error เจียบ ๆ



Case Study: Retry แล้ว escalate ทั้งที่ลูกค้าไม่มีจริง

อาการ

tool ค้น **empty array** หลัง lookup ลูกค้า → agent **retry 3 ครั้ง** → escalate ให้คนจริง ทั้งที่ภายหลังพบว่า "account ลูกค้าไม่มีอยู่จริง" ตั้งแต่แรก

สาเหตุ

tool **ไม่แยก** access failure ออกจาก valid empty result

→ agent เข้าใจว่า "ไม่พบ match" = "ระบบล้มเหลวที่ retry ได้"

✓ วิธีแก้

ให้ tool ตอบ empty result ด้วย `isError: false` + `resultCount: 0` เพื่อให้ agent รู้ว่าคำตอบนี้**สิ้นสุดแล้ว** ไม่ต้อง retry ไม่ต้อง escalate



Exam Traps ที่ต้องระวัง

ตัวหลอก #1 — retry ผลลัพธ์ empty ที่มาจาก query สำเร็จ

× เพราะสับสนว่าเป็นความล้มเหลว — ที่จริงคือคำตอบที่สิ้นสุดแล้ว

ตัวหลอก #2 — ใช้ error message แบบ generic

× ไม่มี structured metadata (`errorCategory` / `isRetryable`) → agent เดาทางกู้คืนไม่ได้

ตัวหลอก #3 — ปฏิบัติกับ business error เหมือน retry ได้

× request เติมยังงี้ก็ fail — การ retry แค่เปลี่ยนเวลา

ตัวหลอก #4 — subagent กลืน error เจียบ ๆ

× คืน empty result เป็น "success" → ดันน้ำไม่รู้ว่ามีปัญหา



Practice Scenario

โจทย์

tool คืน empty array หลัง lookup ลुकค้า · agent retry 3 ครั้งแล้ว escalate ให้คนจริง · ภายหลังพบว่า account ลुकค้า**ไม่มีอยู่จริง**

→ อะไรคือ **root cause** ของความเสียหายนี้?

A. retry limit ต่ำไป เพิ่มเป็น 5 ครั้งจะแก้ได้

B. tool ไม่แยก access failure ออกจาก valid empty result agent จึงมองว่า "ไม่พบ match" เป็น failure ที่ retry ได้

C. เกณฑ์ escalation เชื่อมไป agent ควร retry ให้หมดก่อนถึงคนเข้ามา

D. system prompt ควรสั่ง agent ไม่ให้ retry การ lookup ลुकค้า



หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อเฉลย



เฉลย Practice Scenario

คำตอบที่ถูก — Option B

tool ไม่แยก access failure ออกจาก valid empty result → agent ตีความ query ที่สำเร็จแต่ไม่พบ **match** ว่าเป็นความล้มเหลวของระบบ

ทางแก้: ให้ tool บอกชัดว่า "ไม่มีข้อมูลตรงเงื่อนไข" เป็นคำตอบที่สิ้นสุด (`isError: false` , `resultCount: 0`) ไม่ต้อง retry

ทำไมข้ออื่นผิด

A และ **C** แก้ที่อาการ (จำนวน retry / เกณฑ์ escalation) ทั้งที่ต้นเหตุคือการตีความ empty result ผิด — retry ก็ครั้งก็ยังได้ empty เหมือนเดิม

D กว้างและผิดหลัก เพราะบาง lookup ที่ล้มเหลวจริง (transient) ควร retry ได้ — ปัญหาไม่ใช่ "ห้าม retry เสมอ" แต่คือ tool ต้องแยกประเภทผลลัพธ์ให้ถูก



Build Exercise

โจทย์ฝึก 45 นาที — สรุปสิ่งที่ต้องทำเป็น

1 สร้าง **mock database tool** ที่จำลอง failure mode ได้

2 implement error response ตาม 4 ประเภท พร้อม metadata (`errorCategory` / `isRetryable`)

3 แยก **valid empty result** (`isError: false` , `resultCount: 0`) ออกจาก access failure ให้ชัด

4 เขียน **agent recovery loop** ที่แตกกิ่งตามประเภท error

transient/validation → ลองใหม่ · business/permission → เปลี่ยน workflow หรือ escalate · empty result → หยุดและรายงานว่าไม่พบ

Domain 2 · Task 2.2 — Structured Error Responses ✓



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

tool คืน empty array หลัง lookup ลुकค้า · agent retry 3 ครั้งแล้ว escalate ให้ human agent · ผลวิเคราะห์พบว่า account ลुकค้า**ไม่มีอยู่จริง** · อะไรคือ root cause ของความเสียหายนี้?

A. retry limit ต่ำไป เพิ่มขึ้นเป็น 5 ครั้งจะแก้ปัญหได้

B. tool ไม่แยก access failure ออกจาก valid empty result agent จึงมองว่า "ไม่พบ match" เป็น failure ที่ retry ได้

C. เกณฑ์ escalation เข้มไป agent ควร retry ให้หมดก่อนถึงคนเข้ามา

D. system prompt ควรสั่ง agent ไม่ให้ retry การ lookup ลुकค้า

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option B

tool query แห่่งข้อมูลสำเร็จแล้วแต่ไม่พบ match ซึ่งเป็น valid empty result ไม่ใช่ access failure · เมื่อไม่มี structured metadata แยกสองกรณีนี้ agent จึงตีความทั้งคู่ว่าเป็น failure

ทำไมข้ออื่นผิด

- A — เพิ่ม retry ยิ่งทำให้แย่ง tool ทำงานสำเร็จอยู่แล้ว retry query ที่ไม่พบ match ก็ได้ผลเดิม
- C — ปัญหาไม่ใช่เกณฑ์ escalation valid empty result ไม่ควร retry และไม่ควรถัด escalade เลย
- D — hard-code กฎ retry ราย tool ใน system prompt ปรระแะ generalise ไม่ได้ ทางแก้ที่ถูกคือ structured error metadata



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

refund tool ค้น error: "Refund amount of 750 pounds exceeds the 500 pounds automatic refund limit." · agent retry 3 ครั้ง · ทำไมทุก retry ถึง fail?

- A. retry delay สิ้นไป business rule อาจ update ระหว่าง retry
- B. นี่คือ business error การละเมิด business rule ไม่มีทางแก้ด้วยการ retry เพราะ policy เดิมใช้ทุกครั้ง agent ควร escalate แทน
- C. tool ไม่ได้ค้น isError flag agent จึงไม่รู้ว่าเกิด failure
- D. agent ต้องลดจำนวนเงิน refund อัตโนมัติก่อน retry

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option B

business error (การละเมิด policy / เกินลิมิต) เป็น non-retryable โดยเนื้อแท้ เพราะ business rule เดิมใช้ทุกครั้งที่ลอง · agent ต้องเลือกเส้นทางอื่น ซึ่งโดยทั่วไปคือ escalate ให้ manager

ทำไมข้ออื่นผิด

A — business rule ไม่เปลี่ยนระหว่าง retry ใน session เดียวกัน ลิมิต 500 pounds เป็น policy ไม่ใช่สถานะชั่วคราว

C — ปัญหาไม่ใช่ isError flag แต่คือ metadata ไม่ระบุ isRetryable: false agent จึงตีความว่า retry ได้

D — agent ไม่ควรรูดจำนวน refund เอง ยอดเต็มต้องได้ human approval จึงต้อง escalate



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

error metadata ชุดฟิลด์แบบใดที่ช่วยให้ agent ตัดสินใจ recovery ได้อย่างเหมาะสมกับ tool failure ทุกกรณี?

A. errorCode (integer), errorMessage (string), timestamp (ISO date)

B. errorCategory (transient/validation/business/permission), isRetryable (boolean), description (คำแนะนำ recovery ที่คนอ่านเข้าใจได้)

C. httpStatus (integer), retryAfter (seconds), errorBody (JSON)

D. severity (low/medium/high), impact (string), resolution (string)

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

สามฟิลด์นี้ให้ทุกอย่างที่ agent ต้องใช้: `errorCategory` บอกว่า failure ประเภทไหน, `isRetryable` บอกว่าควร retry หรือเปลี่ยนเส้นทาง, `description` ให้คำแนะนำที่คนอ่านเข้าใจสำหรับ recovery หรือ escalate

ทำไมข้ออื่นผิด

A — error code และ timestamp ไม่บอกว่าควร retry หรือควรทำ recovery action อะไร

C — metadata แบบ HTTP-centric map เข้ากับ MCP tool response ไม่ลงตัว `retryAfter` ช่วยแค่ transient error ไม่ครอบคลุม business/permission

D — ระดับ severity ไม่บอกว่าควร retry high-severity transient อาจ retry ได้ ส่วน low-severity business กลับ retry ไม่ได้



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

ในระบบ multi-agent · web search subagent เจอ timeout ใน 2 จาก 5 sources และมีผลลัพธ์สำเร็จจากอีก 3 sources · subagent ควรทำอะไร?

- A. คืนเฉพาะผลลัพธ์ 3 sources ที่สำเร็จ ราวกับว่าคืนครบทั้ง 5 sources
- B. รายงาน failure ให้ coordinator แล้วทิ้งผลลัพธ์ทั้งหมด
- C. รายงาน partial results จาก 3 sources ที่สำเร็จ พร้อมระบุว่า source 4 และ 5 timeout และยังไม่ได้อ่าน
- D. retry ทั้ง 5 sources ใหม่ตั้งแต่ต้นเพื่อให้ครอบคลุมครบ

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option C

subagent ควรส่ง partial results พร้อมบอกสิ่งที่พยายามทำ · coordinator ต้องการ context: "ค้น 3 จาก 5 sources สำเร็จ · source 4 และ 5 timeout · นี่คือผลลัพธ์บางส่วน" เพื่อให้ตัดสินใจได้อย่างมีข้อมูล

ทำไมข้ออื่นผิด

- A — ปิดบัง error แบบเนียน ๆ ซ่อนข้อมูล failure จาก coordinator ทำให้แยกไม่ออกว่า "ไม่พบข้อมูล" กับ "ค้นไม่ได้"
- B — ทั้งผลลัพธ์ที่สำเร็จทำให้ทั้ง workflow จบเพราะ failure บางส่วน ผล 3 sources ยังมีค่า
- D — retry ทั้ง 5 ทั้งผลสำเร็จของ source 1-3 · ควร retry เฉพาะ 4 และ 5 และลอง local recovery ก่อน propagate



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

tool ได้รับ identifier "order-abc" ทั้งที่ format ที่คาดหวังคือ #NNNNN (เช่น #12345) · tool ควรคืนอะไร?

- A. isError: true, errorCategory: "transient", isRetryable: true พร้อมข้อความให้ retry ภายหลัง
- B. isError: false พร้อม empty result set ระบุว่าไม่พบ order
- C. isError: true, errorCategory: "validation", isRetryable: true พร้อมข้อความระบุ format ที่คาดหวังและขอให้ agent จัดรูปแบบ input ใหม่
- D. isError: true, errorCategory: "business", isRetryable: false พร้อมข้อความให้ escalate

⏸️ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option C

นี่คือ validation error เพราะ format ของ input ผิด · retry ได้เพราะ agent แก้ input ได้ (จัดรูปแบบเป็น #NNNNN แล้วลองใหม่) · description ควรระบุ format ที่คาดหวังเพื่อให้ agent รู้วิธีแก้

ทำไมข้ออื่นผิด

- A** — ไม่ใช่ transient error บริการไม่ได้ล่มชั่วคราว input เองที่ผิดรูป รอแล้ว retry ด้วย input เดิมก็ได้ error เดิม
- B** — ค้น empty result ช้อนปัญหาจริง tool ไม่ได้ค้นแล้วไม่พบ แต่ค้นไม่ได้เลยเพราะ format ผิด ทำให้สับสน access failure กับ empty result
- D** — ไม่ใช่การละเมิด business rule แต่เป็นปัญหา input ที่แก้ได้ ไม่ต้อง escalate